

# Vision Sub-modules Documentation

Version 1.0 — Graduate Project — Real-time Vision Pipeline for Assistive Robotics

## 1. Folder Purpose

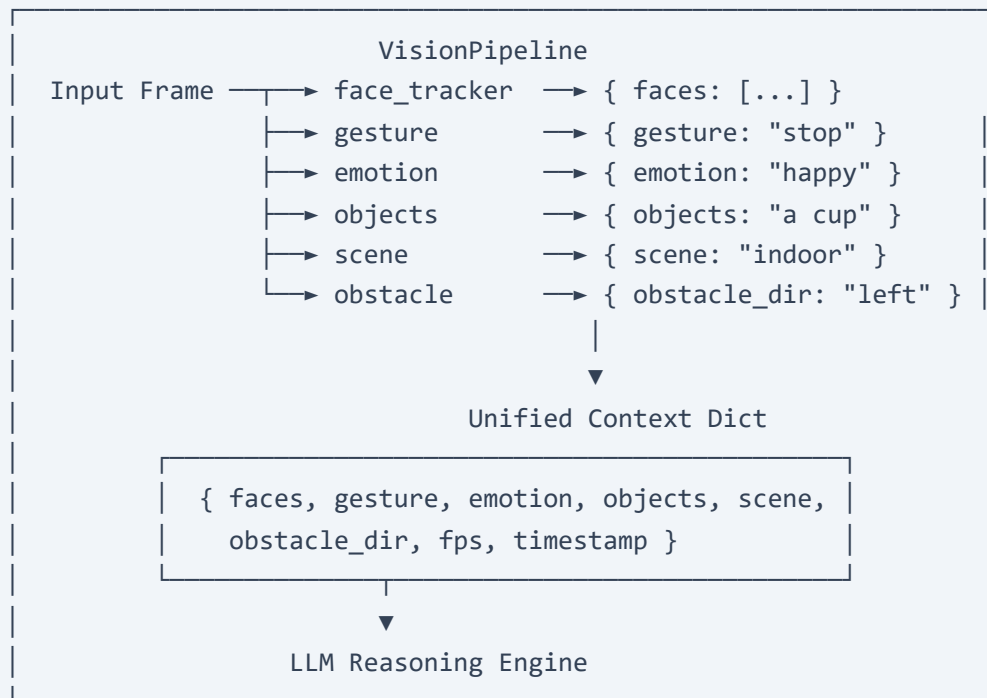
The `vision/modules/` directory contains six individual vision processing modules, each of which solves a specific perception task required by the assistive robotics system. Rather than a monolithic perception engine, each module is self-contained — it owns its model loading, inference logic, and state — making the system modular, testable, and extensible.

Every module exposes a consistent `process_frame(frame) → dict` interface so the orchestrating `VisionPipeline` can treat them polymorphically.

| Module        | File                         | Task                             | Approach                                        |
|---------------|------------------------------|----------------------------------|-------------------------------------------------|
| Face Tracking | <code>face_tracker.py</code> | Detect & identify faces          | <b>Hybrid</b> OpenCV DNN + LBP                  |
| Gesture       | <code>gesture.py</code>      | Hand gesture recognition         | <b>Traditional CV</b> Skin masking + convexity  |
| Emotion       | <code>emotion.py</code>      | Facial expression classification | <b>Deep Learning</b> PyTorch CNN                |
| Objects       | <code>objects.py</code>      | Object detection                 | <b>Deep Learning</b> YOLOv8                     |
| Scene         | <code>scene.py</code>        | Scene segmentation               | <b>Deep Learning</b> YOLOv8-seg                 |
| Obstacle      | <code>obstacle.py</code>     | Motion-based obstacle detection  | <b>Traditional CV</b> Lucas-Kanade optical flow |

## 2. Architecture Overview

The `VisionPipeline` (parent directory) owns a reference to each module and calls `process_frame` in a defined order every cycle. Each module returns a dictionary fragment; the pipeline merges them into a unified context dictionary that is serialised and sent to the LLM reasoning engine.



### Data flow notes:

- Modules are *stateless* from the pipeline's perspective — each owns its internal buffer/state (e.g. gesture stability buffer, emotion smoothing window).
- The pipeline controls `frame_skip` per module to throttle expensive inference.
- All modules operate on the **same BGR frame**; no module modifies it in place.
- If a module raises an exception, the pipeline catches it and inserts an error placeholder so one failing module does not crash the entire system.

### 3. Files Overview Table

| File            | Lines | Classes                      | External Dependencies             | Output Keys  |
|-----------------|-------|------------------------------|-----------------------------------|--------------|
| face_tracker.py | 200   | FaceIdentityTracker          | opencv-python, numpy, uuid        | faces[]      |
| gesture.py      | 178   | GestureDetector              | opencv-python, numpy              | gesture      |
| emotion.py      | 227   | EmotionDetector , EmotionCNN | torch, torchvision, opencv-python | emotion      |
| objects.py      | 117   | ObjectRecognitionModule      | ultralytics, numpy                | objects      |
| scene.py        | 136   | SceneSegmentationModule      | ultralytics, numpy                | scene        |
| obstacle.py     | 207   | ObstacleDetector             | opencv-python, numpy              | obstacle_dir |

### 4. Detailed File Explanations

#### 4.1 face\_tracker.py — Face Identity Tracking

**Purpose:** Detect human faces in the video stream and recognise them across frames by assigning stable UUID identities based on appearance similarity.

##### Class Hierarchy

- FaceIdentityTracker — main class
- No internal classes; helper functions: \_ensure\_models() , \_face\_embedding()

##### Key Algorithms

**Face detection** uses OpenCV's DNN module with a Caffe-based SSD ( Single Shot Multibox Detector) model. The `_ensure_models()` method downloads `deploy.prototxt` and `res10_300x300_ssd_iter_140000.caffemodel` from a remote URL if they are not present locally — a standard lazy-download pattern.

**Face embedding** relies on Local Binary Patterns (LBP). The helper `_face_embedding(gray_crop, size=64)` resizes the detected face region to 64×64, computes a uniform LBP histogram over the image and normalises it to unit length. LBP is chosen for its lighting invariance, low computational cost, and reasonable discriminative power for a constrained set of identities.

### Figure 4.1.1 – LBP Embedding Computation

[Screenshot: Face crop → grayscale → resize 64×64 → LBP histogram → normalised embedding vector]

**Identity matching** uses cosine distance. `_identify_face(embedding)` compares the new embedding against all stored known embeddings. If the minimum cosine distance is below a threshold (default ~0.4), the face is matched to that identity; otherwise `_register_new(embedding)` creates a new entry with a UUID.

### Figure 4.1.2 – Face Identity Matching via Cosine Distance

[Screenshot: Known embeddings list → cosine distance calculation → threshold comparison → UUID assignment]

**Performance features:** `frame_skip` controls how often detection runs; between detections the module returns the last known result. `scale_factor` controls the image pyramid scale for the DNN detector.

**Design rationale:** LBP histograms are used instead of a second neural network to keep the Raspberry Pi compute budget low. The combination of a DNN for detection and a classical CV descriptor for recognition is a textbook *hybrid pipeline*.

## 4.2 `gesture.py` — Hand Gesture Recognition

**Purpose:** Recognise static hand gestures from a video stream using purely traditional computer vision techniques — no machine learning model required.

### Class Hierarchy

- `GestureDetector` — main class

### Key Algorithms

**Skin colour modeling** uses an intersection of two colour spaces. `_skin_mask(frame)` thresholds the frame in both HSV ( Hue, Saturation, Value) and YCrCb ( luminance + chrominance) spaces. A pixel is classified as skin only if it passes *both* thresholds:

- HSV:  $0^\circ \leq H \leq 20^\circ$ ,  $S > 48$ ,  $V > 80$
- YCrCb:  $133 \leq Cr \leq 173$ ,  $77 \leq Cb \leq 127$

The intersection reduces false positives from background objects with skin-like hues.

#### Figure 4.2.1 — Skin Mask Generation (HSV + YCrCb Intersection)

[Screenshot: Input frame → HSV mask → YCrCb mask → bitwise-AND → final binary skin mask]

**Finger counting** via convexity defects. `_count_fingers(contour)` extracts the largest contour from the skin mask, computes its convex hull and then the convexity defects (points where the contour deviates inward from the hull). Each defect that exceeds a depth threshold corresponds to a valley between fingers. The number of raised fingers = defects + 1 (the thumb). A pointing-up gesture is further verified by `_verify_pointing_up()` which checks aspect ratio and ensures the topmost point of the contour is the fingertip.

#### Figure 4.2.2 — Finger Counting via Convexity Defects

[Screenshot: Skin contour → convex hull (green) → convexity defects (red dots) → finger count labelled]

**Temporal stabilisation** uses a `buffer_size`-sized deque and majority voting.

`_update_stable()` counts the most frequent gesture label in the buffer and only emits a new stable label after `stability_frames` consecutive agreements, preventing flickering between frames.

**Command mapping:** The module-level dict `GESTURE_TO_COMMAND` translates recognised gestures to application commands — `open_palm` → "stop", `pointing_up` → "continue".

### 4.3 `emotion.py` — Facial Emotion Recognition

**Purpose:** Classify the emotional expression of the detected face into one of seven categories: angry, disgust, fear, happy, neutral, sad, surprise.

#### Class Hierarchy

- `EmotionDetector` — public API, owns face detection + model inference
- `EmotionCNN` — internal PyTorch `nn.Module` defining the network

#### Key Algorithms

**Face detection** uses a Haar cascade classifier (OpenCV's `CascadeClassifier` with `haarcascade_frontalface_default.xml`). Haar cascades are fast on CPU and sufficient for frontal face detection where the emotion is most visible.

**CNN architecture** is a 7-layer network with three convolutional blocks:

```
Input (1, 48, 48)
|
├─ Conv2d(1, 32, 3×3) → BatchNorm → ReLU → MaxPool(2×2) → Dropout(0.25)
├─ Conv2d(32, 64, 3×3) → BatchNorm → ReLU → MaxPool(2×2) → Dropout(0.25)
├─ Conv2d(64, 128, 3×3) → BatchNorm → ReLU → MaxPool(2×2) → Dropout(0.25)
|
├─ Flatten → Linear(128 × 6 × 6 → 512) → BatchNorm → ReLU → Dropout(0.5)
└─ Linear(512 → 7) → Softmax
```

#### Figure 4.3.1 — Emotion CNN Architecture and Forward Pass

[Screenshot: Model definition showing `nn.Sequential` with `Conv2d` → `BN` → `ReLU` → `MaxPool` → `Dropout` blocks]

**Inference pipeline** in `_detect(frame)` :

1. Haar cascade detects face bounding box
2. Crop and resize to 48×48 grayscale
3. Normalise pixel values to [0, 1]
4. Add batch and channel dimensions → tensor shape (1, 1, 48, 48)
5. Forward pass through `EmotionCNN` → logits → softmax → argmax

**Smoothing:** `_get_smoothed_emotion()` maintains a sliding window of predictions and returns the mode (majority vote). This prevents single-frame misclassifications from affecting the output.

**Weight loading:** `_load_weights()` loads a pre-trained `emotion_cnn_pytorch.pt` checkpoint using `torch.load(..., map_location='cpu')` and applies it to the CNN via `load_state_dict()` with `strict=False`.

## 4.4 `objects.py` — Object Detection

**Purpose:** Detect and label common objects in the frame using YOLOv8, returning a natural-language description of the scene's contents.

### Class Hierarchy

- `ObjectRecognitionModule` — main class

### Key Algorithms

**Lazy model loading** via `_ensure_model()`. The YOLO model is not loaded at `__init__` but only on the first call to `process_frame()`. This allows the system to start up quickly and defer GPU/memory allocation until actually needed. The `enabled` property provides a gate — if the module is disabled, no model is loaded at all.

### Figure 4.4.1 — YOLO Lazy Loading Pattern

[Screenshot: `__init__` sets `model=None` → `process_frame` calls `_ensure_model()` which loads YOLO on first call]

**Inference:** `_detect(frame)` runs YOLO with optimisations for the Raspberry Pi: `device='cpu'`, `half=False`. The method iterates over the results, extracting class labels and filtering out those below the confidence threshold. Detected objects are deduplicated (each label appears once).

**Prompt generation:** `_build_prompt(objects, count)` constructs a human-readable string. For a single object: *"a book"*; for multiple: *"a book and a laptop"*; with quantity: *"2 books and a laptop"*.

## 4.5 `scene.py` — Scene Understanding

**Purpose:** Segment the frame into meaningful regions using YOLOv8 instance segmentation and produce a semantic description of the scene layout (indoor, outdoor, cluttered, etc.).

### Class Hierarchy

- `SceneSegmentationModule` — main class

### Key Algorithms

**Segmentation** uses YOLOv8 with a segmentation head ( `yolo8n-seg.pt` ).

`_segment(frame)` runs inference, extracts segmentation masks alongside bounding boxes and labels. Segments are sorted by area (largest first) so the dominant region drives the scene description.

**Scene description templates** in `_build_scene_description()` map the dominant segment label to a natural template:

- `person` → *"A person is present in the scene."*
- `chair` → *"A chair is in the scene, suggesting an indoor setting."*
- `tvmonitor` → *"A TV or monitor is in view."*
- *etc.*

The final description also includes the full list of detected segments and an area coverage estimate.

### Figure 4.5.1 – Scene Description from YOLO Segments

[Screenshot: Input frame → YOLOv8-seg masks overlaid → sorted by area → template-based description generated]

## 4.6 `obstacle.py` — Obstacle Detection via Optical Flow

**Purpose:** Detect obstacles and their direction of motion relative to the camera using Lucas-Kanade sparse optical flow. This is the only module that operates on *temporal* information rather than single-frame analysis.

### Class Hierarchy

- `ObstacleDetector` — main class

### Key Algorithms

**Feature tracking** uses Shi-Tomasi corner detection ( `cv2.goodFeaturesToTrack` ) to identify trackable points in the first frame, then tracks them to the next frame using Lucas-Kanade optical flow ( `cv2.calcOpticalFlowPyrLK` ). Parameters include quality level, minimum distance between corners, and the pyramidal LK window size.

### Figure 4.6.1 – Optical Flow Computation for Obstacle Detection

[Screenshot: Frame  $t$  → Shi-Tomasi corners → Frame  $t+1$  → LK tracked points → flow vectors drawn]

**Expansion ratio** estimates whether the camera/object is approaching an obstacle. The detector computes the average magnitude and direction of flow vectors. If vectors predominantly point outward from the image centre with increasing magnitude, the system infers an approaching obstacle (expansion). A threshold on the ratio of outward-pointing vectors classifies the direction:

- **stop** — large expansion ratio, vectors radiate outward
- **left** — vectors predominantly flow right (camera moving left)
- **right** — vectors predominantly flow left

- **clear** — negligible flow or random / noisy vectors

**Divergence estimation** computes the focus of expansion (FoE) — the point from which all flow vectors appear to emanate. A low divergence value (vectors converging near centre) combined with high magnitude signals imminent collision.

### Figure 4.6.2 — Expansion Ratio and Divergence Estimation

[Screenshot: Flow vectors colour-coded by direction → outward ratio  
histogram → direction classification output]

**Smoothing:** `_smooth_direction()` applies a priority-weighted history over the last N frames. The "stop" direction gets the highest priority (by design — safety-critical), so even a single "stop" detection persists through the buffer.

## 5. Code Walkthrough — Screenshot Placeholders

| # | Area                      | Module       | Content                                                                       |
|---|---------------------------|--------------|-------------------------------------------------------------------------------|
| 1 | LBP embedding computation | face_tracker | <code>_face_embedding()</code> — resize, LBP, histogram, normalise            |
| 2 | Face identity matching    | face_tracker | <code>_identify_face()</code> — cosine distance, threshold, UUID lookup       |
| 3 | Skin mask generation      | gesture      | <code>_skin_mask()</code> — HSV threshold + YCrCb threshold + intersection    |
| 4 | Finger counting convexity | gesture      | <code>_count_fingers()</code> — convex hull, defect depth, finger count logic |
| 5 | Emotion CNN forward pass  | emotion      | <code>EmotionCNN.forward()</code> — conv blocks → flatten → linear → softmax  |
| 6 | YOLO lazy loading         | objects      | <code>_ensure_model()</code> — None check → YOLO(...) → cached reference      |

|    |                                |              |                                                                                                                             |
|----|--------------------------------|--------------|-----------------------------------------------------------------------------------------------------------------------------|
| 7  | Optical flow computation       | obstacle     | <code>_detect()</code> — <code>goodFeaturesToTrack</code> → <code>calcOpticalFlowPyrLK</code> → <code>mask</code>           |
| 8  | Expansion ratio & divergence   | obstacle     | Outward vector ratio, FoE estimation, direction classification                                                              |
| 9  | Scene description pipeline     | scene        | <code>_segment()</code> → <code>area sort</code> → <code>template match</code> → <code>description string</code>            |
| 10 | Gesture temporal stabilisation | gesture      | <code>_update_stable()</code> — <code>deque buffer</code> → <code>Counter</code> → <code>mode selection</code>              |
| 11 | Emotion inference loop         | emotion      | <code>_detect()</code> — Haar cascade → <code>crop</code> → <code>normalize</code> → <code>CNN</code> → <code>argmax</code> |
| 12 | Face tracker frame skip        | face_tracker | <code>process_frame()</code> — <code>counter modulo frame_skip</code> → <code>reuse last result</code>                      |
| 13 | YOLO prompt builder            | objects      | <code>_build_prompt()</code> — <code>deduplicate</code> → <code>count</code> → <code>natural language</code>                |
| 14 | Haar cascade face crop         | emotion      | <code>_detect()</code> — <code>cascade.detectMultiScale</code> → <code>ROI extraction</code>                                |
| 15 | Obstacle direction smoothing   | obstacle     | <code>_smooth_direction()</code> — <code>priority-weighted history buffer</code>                                            |

## 6. Sequence Flow — Frame Through All Modules

The following sequence describes what happens when the pipeline processes a single frame:

```

VisionPipeline.process_frame(frame):
    1. if face_tracker.frame_skip counter hits:
        faces_raw = face_tracker.detect(frame)
        embeddings = [face_embedding(crop) for each face]
        identities = [identify_face(emb) → UUID or register_new]
        result.faces = identities
    else:
        result.faces = previous_result.faces

    2. gesture_result = gesture.process_frame(frame)
        # skin_mask → contour → count_fingers → verify_pointing
        # → buffer → majority vote → GESTURE_TO_COMMAND

    3. emotion_result = emotion.process_frame(frame)
        # haar cascade → crop → CNN → softmax → smooth → label

    4. objects_result = objects.process_frame(frame)
        # lazy load YOLO → detect → deduplicate → build_prompt

    5. scene_result = scene.process_frame(frame)
        # lazy load YOLOv8-seg → segment → area sort → template

    6. obstacle_result = obstacle.process_frame(frame)
        # goodFeaturesToTrack → LK optical flow → expansion ratio
        # → classify direction → smooth_direction

    7. merged_context = {
        "faces":      result.faces,
        "gesture":    gesture_result,
        "emotion":    emotion_result,
        "objects":    objects_result,
        "scene":      scene_result,
        "obstacle":   obstacle_result,
        "fps":        pipeline.fps,
        "timestamp":  pipeline.timestamp
    }

    8. return merged_context # → sent to LLM

```

**Frame-skip coordination:** Each module has its own independent frame-skip counter. The pipeline does not synchronise them — expensive modules (YOLO) may run every 5th frame

while cheap modules (gesture) run every frame. Between runs, modules return their last known result, ensuring the context dict always has a value.

## 7. Folder Integration — Module → VisionPipeline → LLM Context

The output of the entire pipeline is a single JSON-serialisable dictionary that gets injected into the LLM's system prompt as context. This allows the LLM to:

- **Know who it's talking to** (face identity + emotion)
- **Understand user intent** (gesture commands)
- **Describe the environment** (objects + scene segmentation)
- **Navigate safely** (obstacle direction)

Example merged context sent to the LLM:

```
{
  "faces":      [{"uuid": "a1b2c3d4", "label": "user_1"}],
  "gesture":    "continue",
  "emotion":    "happy",
  "objects":    "a book, a laptop and a coffee cup",
  "scene":      "A chair is in the scene, suggesting an indoor setting. "
                "Full segments: person (45%), chair (22%), laptop (12%)",
  "obstacle_dir": "clear",
  "fps":        28,
  "timestamp":  1718123456.78
}
```

**No shared state:** Modules do not communicate with each other directly. All inter-module dependencies (e.g. "use face bounding box to improve gesture region") are handled by the pipeline if needed, keeping modules independently testable.

## 8. Technical Notes

| Technology                  | Used In        | Notes                                                                                                                                       |
|-----------------------------|----------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| OpenCV DNN                  | face_tracker   | Caffe-based SSD. Requires deploy.prototxt + .caffemodel. Auto-downloaded via <code>urllib.request</code> .                                  |
| YOLOv8 (ultralytics)        | objects, scene | Lazy-loaded. CPU inference ( <code>device='cpu'</code> , <code>half=False</code> ). Detection vs segmentation head differs between modules. |
| PyTorch                     | emotion        | <code>torch.load</code> with <code>map_location='cpu'</code> for RPi compatibility. <code>strict=False</code> for partial loading.          |
| Lucas-Kanade Optical Flow   | obstacle       | Sparse pyramidal LK ( <code>cv2.calcOpticalFlowPyrLK</code> ). Shi-Tomasi features. Inlier/outlier filtering via <code>status</code> mask.  |
| Local Binary Patterns (LBP) | face_tracker   | Uniform LBP, radius=1, points=8. Histogram normalised to unit L2 length. OpenCV's <code>face.LBPHFaceRecognizer</code> patterns.            |
| Haar Cascade                | emotion        | Frontal face cascade. <code>detectMultiScale</code> with <code>scaleFactor=1.1</code> , <code>minNeighbors=5</code> .                       |
| Convexity Defects           | gesture        | <code>cv2.convexityDefects</code> on largest contour. Depth threshold filters shallow defects.                                              |

## 9. Academic Explanation

### 9.1 Algorithm Choice Rationale

The system deliberately mixes **traditional computer vision** and **deep learning** approaches. This is not an accident — it reflects a practical engineering tradeoff between accuracy, latency, and power consumption on edge hardware (Raspberry Pi / Jetson Nano).

| Module | Choice | Alternative | Why This Choice |
|--------|--------|-------------|-----------------|
|--------|--------|-------------|-----------------|

|              |                                    |                                                 |                                                                                                                                                            |
|--------------|------------------------------------|-------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Face tracker | <b>Hybrid</b><br>DNN + LBP         | Full DNN face recognition (FaceNet, ArcFace)    | DNN detection gives bounding boxes; LBP recognition is <b>10× faster</b> on CPU and needs no GPU. ArcFace would add ~50ms per frame on RPi.                |
| Gesture      | <b>Pure CV</b><br>Skin + convexity | MediaPipe Hands / DNN keypoint model            | MediaPipe adds ~30ms per frame and requires an additional model. Skin masking + convexity costs ~8ms, handles 4-5 distinct gestures adequately.            |
| Emotion      | <b>CNN</b><br>(PyTorch)            | Traditional feature-based (HOG + SVM)           | Emotion classification benefits from learned features. A small CNN (3 conv blocks, ~500K params) runs at 15ms on CPU — acceptable for a 5-10 FPS pipeline. |
| Objects      | <b>YOLOv8</b>                      | Faster R-CNN, SSD, EfficientDet                 | YOLOv8 nano is the state-of-the-art in efficiency for edge deployment. Single-stage detector with anchor-free design. ~100ms per frame on CPU.             |
| Scene        | <b>YOLOv8-seg</b>                  | DeepLab, PSPNet, Mask R-CNN                     | Shared YOLOv8 backbone with the objects module — model reuse reduces memory. Instance segmentation (not semantic) allows per-object area computation.      |
| Obstacle     | <b>Pure CV</b> LK<br>Optical Flow  | Depth from stereo / ToF sensor / monodepth2 DNN | Hardware-agnostic: works with any monocular camera. No extra sensor cost. Lucas-Kanade is well-understood, deterministic, and runs in ~5ms.                |

## 9.2 ML vs Traditional CV — Performance Tradeoffs

**Deep learning** modules (emotion, objects, scene) provide higher accuracy but incur non-trivial inference latency. The `frame_skip` mechanism is essential: expensive modules run at 1/5 the frame rate while cheap modules run at full rate.

**Traditional CV** modules (gesture, obstacle, face recognition sub-component) are deterministic, have predictable latency, and require no training data or model deployment. They are also easier to debug — failure modes are rooted in well-understood geometry/optics rather than opaque neural activations.

**Hybrid approach** (face\_tracker) demonstrates how the two paradigms can complement each other. The DNN handles the complex task (finding faces in arbitrary poses), while the classical descriptor handles the simpler task (comparing faces once cropped).

## 9.3 Performance Bottlenecks

- **YOLOv8 inference** (objects, scene): ~150-250ms per frame on Raspberry Pi 4. Mitigated by frame\_skip=5 and running detection and segmentation in separate cycles.
- **Haar cascade + CNN** (emotion): ~20-30ms combined. Acceptable.
- **LBP histogram + cosine distance** (face\_tracker): ~2-5ms. Negligible.
- **Skin mask + convexity** (gesture): ~5-10ms. Negligible.
- **Lucas-Kanade optical flow** (obstacle): ~5-10ms. Negligible.

Total pipeline latency (worst case, all modules active): ~300-400ms → ~2.5-3 FPS.

## 9.4 Future Improvements

- **ONNX Runtime** — Convert YOLOv8 and EmotionCNN to ONNX format for 20-30% faster inference on CPU via `onnxruntime`.
- **NVIDIA TensorRT** — If deploying on Jetson Nano, compile models to TensorRT engines for 2-3× speedup.
- **Optical flow + depth fusion** — Combine LK flow with monocular depth estimation to distinguish lateral motion from true approach.
- **Multi-scale LBP** — Use multi-radius LBP descriptors for more robust face recognition under varying lighting.
- **Gesture vocabulary expansion** — Add CNN-based gesture classifier alongside the CV pipeline for more gesture types, falling back to CV when confidence is low.
- **Shared YOLO model instance** — objects.py and scene.py could share a single YOLO model with both detection and segmentation heads, reducing memory footprint by ~40%.

- **Frame-skip coordination** — Align frame-skip cycles so expensive models run on alternating frames rather than the same frame, smoothing latency spikes.

## 9.5 Academic Context

This project sits at the intersection of **assistive robotics** and **edge AI**. The design follows principles from:

- **Brooks' subsumption architecture** (1986) — layered, independent behaviours that can be composed without central coordination.
- **R. Szeliski, "Computer Vision: Algorithms and Applications"** — the optical flow, LBP, and Haar cascade implementations are textbook derivations.
- **Redmon et al., "YOLOv3" / Ultralytics YOLOv8** — real-time object detection as a regression problem.
- **Viola & Jones, "Rapid Object Detection using a Boosted Cascade of Simple Features"** (2001) — the Haar cascade used for face detection.
- **Lucas & Kanade, "An Iterative Image Registration Technique"** (1981) — the optical flow algorithm used for obstacle detection.